

Gradient descent

best performance

↳ optimization

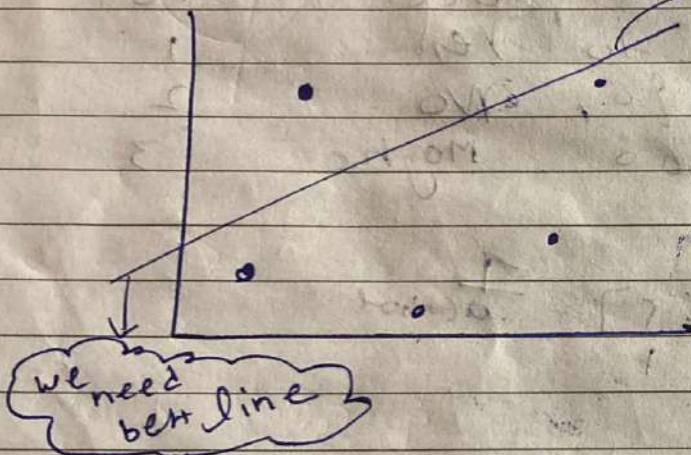
Algorithm

Gradient descent is a first-order iterative optimization algorithm for finding local minimum of a differentiable function.

notation

$$y = mx + b$$

↓
up niche



vo line jo y direction mein shre kam ghti

Here

↓ minimum

$$LF = \sum_{i=1}^n (y_i - \hat{y}_i)$$

$$\hat{y}_i = mx_i + b$$

$$L = \sum_{i=1}^n (y_i - mx_i - b)^2$$

fixed (under y_i) variable (under m, b)
 fixed (under x_i) variable (under m, b)

$$L(m, b) =$$

variable
mein ager
changes hoga
to LF change
hoga



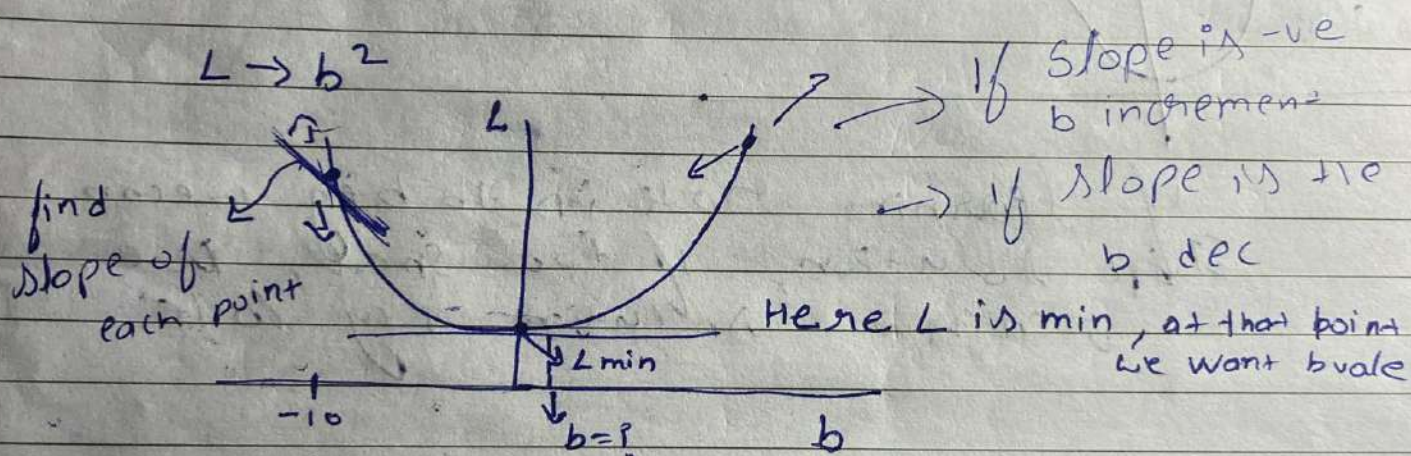
Let m be fixed

$$m = 78.35$$

$$L = \sum_{i=1}^n (y_i - 78.5x_i - b)^2$$

$\uparrow$
variable

now L depends on b : if m is fixed



Step $\rightarrow$ 1 select random b

$$b = -10$$

$$b_{\text{new}} = b_{\text{old}} - \text{slope}$$

$$\text{let } b_{\text{old}} = -10$$

$$\text{slope} = 50$$

$$\text{if } b = 10$$

$$\text{slope} = 50$$

$$b_{\text{new}} = 10 - 50$$

$$= -40$$

$$b_{\text{new}} = -10 - (-50)$$

$$\Rightarrow 40$$

To find slope

$$n=5$$

$$y = u^2 + 2$$

$$\frac{dy}{du} = 2u$$

$$\frac{dy}{du} = 10$$

point u is slope

10

bold = -10
slope = -50

learning rate $\rightarrow 0.01$

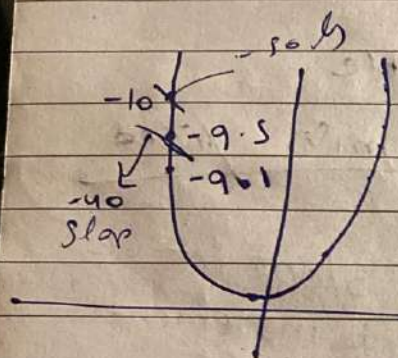
PAGE NO.

0.01

0.001

$$b_{\text{new}} = \text{bold} - \eta \text{slope}$$

$$\begin{aligned} b_{\text{new}} &= -10 - (\eta \times -50) \\ &= -10 + (0.01 \times 50) \\ &= -10 + (0.5) \\ &= -9.5 \end{aligned}$$



$$\begin{aligned} b_{\text{new}} &= -9.5 - (\eta \times -90) \\ b_{\text{new}} &= -9.1 \end{aligned}$$

If learning rate is too big, we can skip solution, if too small, we reach solution very slowly.

When to Stop

1]

$$b_{\text{new}} - \text{bold} \leq 0.0001$$

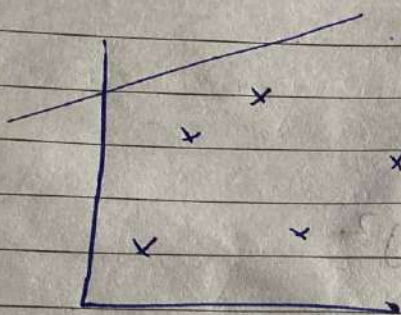
$\rightarrow$ end it

2] Iteration epochs (1000)

$\uparrow$ depends

let's consider we know m
 $m = 78.5$

PAGE NO.:



Step 1 $\rightarrow$ Start with random value of b

let $b = b = 0$

for i in epochs () :

$\eta = 0.01$

$b_{\text{new}} = b_{\text{old}} - \eta \text{ slope}$

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\downarrow$$

$$L = \sum_{i=0}^n (y_i - \hat{y}_i)^2$$

$$\frac{dL}{db} = \frac{d}{db} \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right)$$

$$= \frac{d}{db} \sum_{i=1}^n (y_i - m x_i - b)^2$$

$$2 \sum_{i=1}^n (y_i - m x_i - b) (-1)$$

$$\Rightarrow -2 \sum_{i=1}^n (y_i - m x_i - b)$$

let

$$L = \sum_{i=1}^n (y_i - y_i^*)^2$$

$$\frac{dL}{db} = \frac{d}{db} \sum_{i=1}^n (y_i - mx_i - b)^2$$

$$= -2 \sum_{i=1}^n (y_i - mx_i - b) \quad (1)$$

$$\Rightarrow -2 \sum_{i=1}^n (y_i - mx_i - b)$$

to find slope of any point input here mx
to value

for previous example $m = 79.35$
if take $b = 0$ as random the
Slope will be

$$\Rightarrow -2 \sum_{i=1}^n (y_i - mx_i - b)$$

$$\Rightarrow -2 \sum_{i=1}^n (y_i - 79.35x_i - 0)$$

$$b_{new} = b_{old} = \frac{1}{n} \text{ Slope}_{b=old}$$

✓ 1/ learning rate too low epochs gets over
4 best line not fetched

1/ learning rate too high, skips optimal solution, jumps solution

Setting learning rate depends on hyperparameters (trial & error)

already assumed

$$b = b_{old} - \eta \text{ slope}$$

independent
of Any
algorithm

slope if find by diff with respect to loss function,

eg:-

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\frac{dL}{db} = 2 \sum_{i=1}^n (y_i - m x_i - b)$$

for
linear regression

$$\frac{dL}{db} = -2 \sum_{i=1}^n (y_i - m x_i - b)$$

Here can be any loss function then we will diff it by w.r.t. with respect to b

When we are not taking m as fixed, we find both m & b together

Let,

$$\eta = 0.01$$

$m = 1$ } take random number
 $b = 0$ }

$$\text{Epochs} = 100$$

for i in range (Epochs)

$$b_{\text{new}} = b_{\text{old}} - \eta \text{ Slope}$$

$$m_{\text{new}} = m_{\text{old}} - \eta \text{ Slope}$$

This slope depends

on $\frac{\partial L}{\partial b}$ with diff w.r.t to b

This slope depends

on $\frac{\partial L}{\partial m}$ with diff w.r.t to m

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\frac{\partial L}{\partial m} = 2 \sum_{i=1}^n (y_i - m x_i - b)$$

$$\frac{\partial L}{\partial m} = -2 \sum_{i=1}^n (y_i - m x_i - b) x_i$$

for m slope

for b Slope

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$L = \sum_{i=1}^n (y_i - m x_i - b)^2$$

$$\frac{\partial L}{\partial b} = 2 \sum_{i=1}^n (y_i - m x_i - b)$$

$$\frac{\partial L}{\partial b} = -2 \sum_{i=1}^n (y_i - m x_i - b)$$

We learned Gradient Descent only for
1 feature & 1 target
n dim inputs

Eg: x_1 (Cgpa) x_2 (LPA)
 $x_{11} \rightarrow 8.1$ $x_{12} \rightarrow 9.3$ 3.2
 $x_{21} \rightarrow 7.5$ $x_{22} \rightarrow 9.5$ 3.5

$$y = \underset{\substack{\uparrow \\ \text{Intercept}}}{\beta_0} + \beta_1 x_1 + \beta_2 x_2 \quad (19)$$

1) Random Values, select random values for $\beta_0, \beta_1, \beta_2$

$$\beta_0 = 0$$

$$\beta_1, \beta_2 = 1$$

2) Epochs $\rightarrow 100$, $\eta = 0.1$

$$\beta_0 \rightarrow \beta_0 - \eta \text{ slope} \rightarrow \frac{dL}{d\beta_0}$$

$$\beta_1 \rightarrow \beta_1 - \eta \text{ slope} \rightarrow \frac{dL}{d\beta_1}$$

$$\beta_2 \rightarrow \beta_2 - \eta \text{ slope} \rightarrow \frac{dL}{d\beta_2}$$

we may take for better simplicity

PAGE NO.:

$$LF = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad n=2$$

↓
let's expand

$$LF = \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \beta_2 x_{i2})^2 + (y_1 - \beta_0 - \beta_1 x_{11} - \beta_2 x_{12})^2$$

$$\frac{dL}{d\beta_0} = \frac{1}{n} \sum_{i=1}^n 2(y_i - \beta_0 - \beta_1 x_{i1} - \beta_2 x_{i2}) + 2(y_1 - \beta_0 - \beta_1 x_{11} - \beta_2 x_{12})$$

$$\Rightarrow -2 \sum_{i=1}^n y_i$$

$$\boxed{\frac{dL}{d\beta_0} \Rightarrow \frac{-2}{n} [(y_1 - \hat{y}_1) + (y_2 - \hat{y}_2)]}$$

↓
So for any n

$$\frac{dL}{d\beta_0} \Rightarrow \frac{-2}{n} [(y_1 - \hat{y}_1) + (y_2 - \hat{y}_2) + (y_3 - \hat{y}_3) + \dots]$$

$$\boxed{\frac{dL}{d\beta_0} \Rightarrow \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)}$$

for $\beta_0 = \beta_0 - n \text{ slope}$

$$\frac{dL}{d\beta_1} \rightarrow \beta_1 = \beta_1 - n \text{ slope}$$

↓ ?
diff with respect to β_1 to its loss function

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\frac{dL}{d\beta_1} = \frac{1}{n} \sum_{i=1}^n [(y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2]$$

$$\frac{dL}{d\beta_1} = \frac{1}{n} \sum_{i=1}^n [(y_1 - \beta_0 - \beta_1 x_{11} - \beta_2 x_{12})^2 + (y_2 - \beta_0 - \beta_1 x_{21} - \beta_2 x_{22})^2]$$

$$\frac{dL}{d\beta_1} = \frac{1}{n} [(y_1 - \hat{y}_1) x_{11} + (y_2 - \hat{y}_2) x_{21} + (y_3 - \hat{y}_3) x_{31} + \dots]$$

$$\frac{dL}{d\beta_1} = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{i1}$$

$$\frac{dL}{d\beta_2} = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{i2}$$

$$\frac{dL}{d\beta_m} = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{im}$$

PAGE NO. 5
→ randomly selects rows to update
Stochastic Gradient Descent

↓
we use this Gradient Descent method because it is efficient for big data because in batch gradient descent it will do computation for each input

Eg →

$$L = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) u_i$$

1 epoch = 1 update in batch

↓
it will be calculate for columns

pure data define K batch & update each column

In SGD, each update on each rows

↓ frequent updates, we will reach answer early in less epochs

→ faster Convergence

→ Hardware Support (can run of small computer)

→ Not steady: Answers / when run again again diff values comes because they pick from different rows

It is not necessary that at every epoch we get best solution than previous because of random inputs

for intercept

$$\text{intercept} = \text{intercept} + (\text{len}^{\text{inter_slope}})$$

$$\text{inter_slope} = -2 * (y[\text{id}x] - y_{\text{hat}})$$

$$\text{id}x = \text{np.random.randint}(1, 10)$$

$$y_{\text{hat}} = \text{np.dot}(x[\text{id}x], \text{self.coef-}) + \text{self.intercept}$$

on coef-

$$\text{coef-} = \text{coef-} + (\text{len}^{\text{coef-slope}})$$

$$\text{coef-slope} = -2 * \text{np.dot}(y[\text{id}x] - y_{\text{hat}}, x[\text{id}x])$$

When to use SGD

→ Big data

→ Non convex function → there are local/global minima

Learning Schedule $\rightarrow$ to reduce lr by running epochs, making separate function for it

$t_0, t_1 = 5, 50$

def learning_rate(t):

return $t_0 / (t_1 - t_0 + t)$

for i in range(epochs):

for j in range(x.shape[0]):

$lr = \text{learning_rate}(i * x.\text{shape}[0] + j)$



Mini Batch Gradient Descent



lies between batch GD & Stochastic GD

It uses Batch sizes of random rows to ~~update~~ update intercept & coef

dekha hai ki Mini Batch GD jisse ki job slow, or small datasets ki liye hota hai

Jab ham SGD karte hai toh fast hai but hokey anywhere data hai (flat near about answer, not accurate)

Minibatch GD stable, not noisy solution provide karta hai.